i want to add fog back into my project and have it controllable either through scale or just a simple toggle, here is my current .frag file:

```
#version 460

in vec3 SpotDir;
in vec3 LightDir;
in vec3 ViewDir;
in vec2 TexCoord;

layout (location = 0) out vec4 FragColor;
layout (binding = 0) uniform sampler2D DiffuseTex;
layout (binding = 1) uniform sampler2D NormalMapTex;
layout (binding = 2) uniform sampler2D DiffuseTex2;


uniform struct MaterialInfo{
    vec3 Kd;
    vec3 Ka;
    vec3 Ks;
    float Shininess;
}Material;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct FogInfo{
    float MaxDistance;
    float MinDistance;
    vec3 Colour;
}Fog;

const int levels = 5;
const float scaleFactor = 1.0/levels;



vec3 blinnPhong(vec3 n) {
    vec3 diffuse = vec3(0.0f);
    vec3 spec = vec3(0.0f);

    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Light.La * texColour;
    vec3 s = normalize(LightDir);
    //vec3 s = normalize(Light.Position.xyz - position);

    float sDotN = max(dot(s, n), 0.0);

    diffuse = texColour * sDotN;

    if (sDotN > 0.0) {
        vec3 v = normalize(ViewDir);
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }
    return ambient + (diffuse + spec) * Light.L;
}



vec3 blinnPhongSpot(vec3 n)
{
    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Spot.La * texColour;

    vec3 s = normalize(LightDir);  // tangent-space light dir
    vec3 v = normalize(ViewDir);

    float cosAng = dot(normalize(-s), normalize(SpotDir));

    float outer = cos(Spot.Cutoff);                // outer cone edge
```

```
    float inner = cos(Spot.Cutoff * 0.85);          // inner cone edge

    float spotEdge = smoothstep(outer, inner, cosAng);
    float spotCore = pow(max(cosAng, 0.0), Spot.Exponent);
    float spotScale = spotEdge * spotCore;

    if (spotScale <= 0.001)
        return ambient;

    float sDotN = max(dot(s, n), 0.0);
    vec3 diffuse = texColour * sDotN;

    vec3 spec = vec3(0.0);
    if (sDotN > 0.0) {
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }

    return ambient + spotScale * Spot.L * (diffuse + spec);
}


void main() {
//fog stuff
    //float distance = abs(Position.z);
    //float fogFactor = (Fog.MaxDistance - distance) / (Fog.MaxDistance
- Fog.MinDistance);
    //fogFactor = clamp(fogFactor, 0.0, 1.0);
    //vec3 shadeColour = blinnPhong(Position, normalize(Normal));
    //vec3 Colour = mix(Fog.Colour, shadeColour, fogFactor);  // --
remember to put Colour instead of shadeColour when i want fog back


//blinnphong
    vec3 normal = texture(NormalMapTex, TexCoord).xyz;
    normal = normal * 2.0 - 1.0;
    normal = normalize(normal);
    FragColor = vec4(blinnPhongSpot(normal), 1.0);
}

and my vert file:
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;
layout (location = 2) in vec2 VertexTexCoord;
layout (location = 3) in vec4 VertexTangent;

out vec3 SpotDir;
out vec3 LightDir;
out vec3 ViewDir;
out vec2 TexCoord;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 ProjectionMatrix;
uniform mat4 MVP;

out vec3 LightIntensity, n, pos;

void main()
{
    vec3 N = normalize(NormalMatrix * VertexNormal);
    vec3 T = normalize(NormalMatrix * vec3(VertexTangent.xyz));

    T = normalize(T - N * dot(N, T));

    vec3 B = cross(N, T) * VertexTangent.w;

    // Columns are T, B, N
    mat3 TBN = mat3(T, B, N);
    mat3 invTBN = transpose(TBN);
```

```
    vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    vec3 lightDir;
    if (Light.Position.w == 0.0)
        lightDir = normalize(Light.Position.xyz);       // directional
    else
        lightDir = Light.Position.xyz - Position;       // point light
    LightDir = invTBN * lightDir;

    ViewDir = invTBN * normalize(-Position);
    SpotDir = invTBN * normalize(Spot.Direction);

    TexCoord = VertexTexCoord;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Thought for 11s

Nice — you're *almost* there. The main thing missing for fog is a **per-fragment distance value** (usually view-space position), because your `.frag` currently has no `Position` input to base fog on.

Below is a clean drop-in way to add **fog with a toggle** *and* an optional **scale control** (so you can "intensify" fog without changing your scene scale).

Updated `*.vert` (adds `ViewPos` output)

```glsl
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;
layout (location = 2) in vec2 VertexTexCoord;
layout (location = 3) in vec4 VertexTangent;

out vec3 SpotDir;
out vec3 LightDir;
out vec3 ViewDir;
out vec2 TexCoord;

out vec3 ViewPos; // <-- for fog distance

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 MVP;

void main()
{
    vec3 N = normalize(NormalMatrix * VertexNormal);
    vec3 T = normalize(NormalMatrix * vec3(VertexTangent.xyz));

    T = normalize(T - N * dot(N, T));
    vec3 B = cross(N, T) * VertexTangent.w;

    mat3 TBN = mat3(T, B, N);
    mat3 invTBN = transpose(TBN);

    // View-space position
    vec3 positionVS = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;
    ViewPos = positionVS;

    vec3 lightDir;
    if (Light.Position.w == 0.0)
        lightDir = normalize(Light.Position.xyz);       // directional
    else
        lightDir = Light.Position.xyz - positionVS;      // point light (view-space)
```

glsl

```glsl
    ViewDir  = invTBN * normalize(-positionVS);
    SpotDir  = invTBN * normalize(Spot.Direction);

    TexCoord = VertexTexCoord;
    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Updated `*.frag` (fog toggle + scale)

```glsl
#version 460

in vec3 SpotDir;
in vec3 LightDir;
in vec3 ViewDir;
in vec2 TexCoord;

in vec3 ViewPos; // <-- from vertex shader

layout (location = 0) out vec4 FragColor;

layout (binding = 0) uniform sampler2D DiffuseTex;
layout (binding = 1) uniform sampler2D NormalMapTex;
layout (binding = 2) uniform sampler2D DiffuseTex2;

uniform struct MaterialInfo{
    vec3 Kd;
    vec3 Ka;
    vec3 Ks;
    float Shininess;
}Material;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct FogInfo{
    float MaxDistance;
    float MinDistance;
    vec3 Colour;
}Fog;

// Fog controls (simple + robust)
uniform int   FogEnabled;   // 0 = off, 1 = on
uniform float FogScale;     // 1.0 = normal, >1 stronger fog, <1 weaker

vec3 blinnPhongSpot(vec3 n)
{
    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;
    vec3 ambient = Spot.La * texColour;

    vec3 s = normalize(LightDir);
    vec3 v = normalize(ViewDir);

    float cosAng = dot(normalize(-s), normalize(SpotDir));

    float outer = cos(Spot.Cutoff);
    float inner = cos(Spot.Cutoff * 0.85);

    float spotEdge = smoothstep(outer, inner, cosAng);
    float spotCore = pow(max(cosAng, 0.0), Spot.Exponent);
    float spotScale = spotEdge * spotCore;

    if (spotScale <= 0.001)
        return ambient;

    float sDotN = max(dot(s, n), 0.0);
    vec3 diffuse = texColour * sDotN;

    vec3 spec = vec3(0.0);
    if (sDotN > 0.0) {
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }

    return ambient + spotScale * Spot.L * (diffuse + spec);
}

float fogFactorLinear(float dist)
{
    // 1 = fully shaded, 0 = fully fog
    float f = (Fog.MaxDistance - dist) / (Fog.MaxDistance - Fog.MinDistance);
    return clamp(f, 0.0, 1.0);
}

void main()
{
    vec3 normal = texture(NormalMapTex, TexCoord).xyz;
    normal = normalize(normal * 2.0 - 1.0);

    vec3 litColour = blinnPhongSpot(normal);

    if (FogEnabled != 0)
    {
        float f = fogFactorLinear(dist);
        vec3 finalColour = mix(Fog.Colour, litColour, f);
```

```
        FragColor = vec4(finalColour, 1.0);
    }
    else
    {
        FragColor = vec4(litColour, 1.0);
    }
}
```

What to set from C++ (minimal)

You now need to set **two extra uniforms** somewhere you already set your fog/light uniforms:

- `FogEnabled` (int): `0` or `1`
- `FogScale` (float): start at `1.0f`

Example intent (adapt to your `setUniform` helpers):

```
C++
```

And you already have:

```
prog.setUniform("Fog.MinDistance", ...);
C++
```

Tuning tips (so it doesn't look rubbish)

- Start with something like:
  - `Fog.MinDistance = 10.0f`
  - `Fog.MaxDistance = 60.0f`
  - `FogScale = 1.0f`
- If fog is *barely visible*, increase `FogScale` to `1.5–3.0`.
- If fog is *too heavy*, lower `FogScale` to `0.5–0.8`.

If you want, I can also give you an **exponential fog** version (often looks nicer than linear), but this linear one is the easiest to control and matches your existing `MinDistance/MaxDistance` setup.

can you help me with what precisely i need to do to my .cpp file for this

Yep — here's *precisely* what you need to do in your `.cpp` to drive the new fog uniforms **without changing your existing fog struct code**.

I'm going to assume your shader program wrapper supports the usual overloads like `setUniform(name, int)` / `setUniform(name, float)` / `setUniform(name, vec3)` (same way you're already doing `Material.Shininess`, `Spot.Cutoff`, etc.).

---

1) Add a couple of variables to your scene class

In your `SceneBasic_Uniform` class (header or top of `.cpp` as members), add:

```
bool fogEnabled = true;     // toggle
C++
```

If you want keyboard scale control too:

```
C++
```

---

2) Set the new uniforms every frame before you draw

Wherever you currently set your light/fog uniforms before rendering (often in `render()` right before drawing meshes), add these lines:

```
C++
```

That's it for the *new* uniforms.

Keep your existing fog struct uniforms as well, e.g.:

```cpp
prog.setUniform("Fog.MinDistance", 10.0f);
```

Important

Do this **after** `prog.use()` (or whatever your wrapper uses) and **before** you draw anything using that shader.

---

3) Add a keypress toggle (edge-triggered)

Put this in your `update(float t)` (or wherever you already handle input like your E key colour cycling):

Example: press **F** to toggle fog.

```cpp
GLFWwindow* win = glfwGetCurrentContext();

// Edge-trigger toggle
bool fDown = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
if (fDown && !fKeyWasDown) {
```

---

4) Optional: Control fogScale with keys

Example: **Up/Down arrow** adjusts intensity:

```cpp
bool upDown = (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS);
if (upDown && !upWasDown) {
    fogScale += 0.1f;
    if (fogScale > 5.0f) fogScale = 5.0f;
}
upWasDown = upDown;

bool downDown = (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS);
if (downDown && !downWasDown) {
    fogScale -= 0.1f;
```

```cpp
downWasDown = downDown;
```

If you'd rather it be *held* to smoothly change, do:

```cpp
if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)   fogScale += 0.5f * dt;
```

(You'll need `dt` available and `#include <glm/common.hpp>` or just manual clamp.)

---

5) Common "why isn't it working?" checklist

- You **must** set `FogEnabled` and `FogScale` on the **same shader program** that the fog code is in (your current `.frag` / `.vert` pair).
- Make sure `prog.use()` happens before `setUniform(...)`.
- If you have multiple shader programs (e.g., one for plane, one for model), you need to set these fog uniforms on **each program** that should fog.

If you paste me the bit of your `.cpp` render loop where you set uniforms + draw

... I can tell you *exactly* where to drop the lines in your specific file (and whether you need to do it once or per-program).

i dont see the fog at all.

heres my cpp
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>

```
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    //C1Mesh = ObjMesh::load("media/C1-10P_obj.obj", true);
    //LightsaberMesh = ObjMesh::load("media/Lightsaber_01.obj", true);
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    prog.use();
    glEnable(GL_DEPTH_TEST);

    glEnable(GL_STENCIL_TEST);
    glClearStencil(0);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    //fog
    prog.setUniform("Fog.MinDistance", 10.0f);
    prog.setUniform("Fog.MaxDistance", 60.0f);
    prog.setUniform("Fog.Colour", glm::vec3(0.6f, 0.7f, 0.8f));


    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);


    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");

    //BladeDiffuseTexture =
Texture::loadTexture("media/texture/white.jpg");
    WorkbenchDiffuseMap =
```

```
Texture::loadTexture("media/texture/workbench.png");


    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);


}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;   // toggle once
        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();

            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
        //fog stuff
        if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
        {
            fogEnabled = !fogEnabled;   // toggle once
        }
        F_Pressed = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    render();
```

```
}

void SceneBasic_Uniform::render()
{

    prog.use();

    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT |
GL_STENCIL_BUFFER_BIT);

    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);


    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    prog.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    prog.setUniform("FogScale", fogScale);


    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);
    sky.render();




    prog.use();
    //// Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
    plane.render();

    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));

    setMatrices();
    LightsaberMesh->render();
```

```
    // Blade
    if (bladeOn)
    {
        glm::mat4 bladeBase = mat4(1.0f);
        bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f,
0.0f, 0.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f,
0.0f, 1.0f));

        // -------- Pass 1: White core (writes stencil = 1 where the blade is)
--------
        bladeEmissive.use();

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        // Stencil: always write 1 where fragments pass depth test
        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
        bladeEmissive.setUniform("Intensity", 10.0f);
        bladeEmissive.setUniform("RimPower", 5.0f);
        bladeEmissive.setUniform("RimIntensity", 0.25f);

        bladeEmissive.setUniform("Alpha", 1.0f);
        bladeEmissive.setUniform("CoreAlpha", 1.0f);

        BladeMEsh->render();

        // -------- Pass 2: Coloured glow shell (only OUTSIDE the core
silhouette) --------
        glEnable(GL_BLEND);
        glBlendFunc(GL_ONE, GL_ONE);

        // Don't write depth for glow, but DO depth test so hilt blocks it
        glDepthMask(GL_FALSE);

        // Stencil: only draw where stencil != 1 (i.e., outside the core)
        glStencilMask(0x00);
        glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

        // Optional but usually improves the "wrap" look:
        // render only backfaces of the enlarged shell so it appears as an
outline
        glEnable(GL_CULL_FACE);
        glCullFace(GL_FRONT);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f * 1.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        bladeEmissive.setUniform("Intensity", 2.5f);
        bladeEmissive.setUniform("RimPower", 2.0f);
        bladeEmissive.setUniform("RimIntensity", 1.0f);

        bladeEmissive.setUniform("Alpha", 0.55f);
        bladeEmissive.setUniform("CoreAlpha", 0.0f);

        BladeMEsh->render();

        // -------- Restore state --------
        glCullFace(GL_BACK);
        glDisable(GL_CULL_FACE);

        glDepthMask(GL_TRUE);
        glDisable(GL_BLEND);

        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 0, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    }
```

```
}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
}
```

Thought for 23s ›

Looking at your `.cpp`, the way you're *setting* fog uniforms is fine. When fog "does nothing" in this situation, it's almost always one of these:

1. **Your shader on disk doesn't actually contain the fog code / new uniforms** (so `setUniform("FogEnabled"... )` silently does nothing because the uniform doesn't exist / got optimised out).
2. **Fog range is way too gentle for your scene** (Min/Max means you're basically not fogging anything noticeable).
3. `fogEnabled` / `fogScale` **aren't initialised how you think** (e.g., `fogEnabled` defaults to false, `fogScale` is 0, etc.).
4. **You're expecting fog on the skybox / blade** (it won't show there: skybox uses `skyboxShader`, blade uses `bladeEmissive`).

Below is a very "precise" set of changes + a smoke test that will *force* you to see fog if the shader is wired correctly.

A) Make sure these are initialised (in your class members)

In `SceneBasic_Uniform` (header), ensure:

```cpp
bool fogEnabled = true;  // start ON so you don't rely on toggle
```

If `fogEnabled` / `fogScale` are not members and are uninitialised locals, fog can easily end up "off" forever.

B) Use aggressive fog values (so you can actually see it)

In `initScene()`, temporarily change to:

```cpp
prog.setUniform("Fog.MinDistance", 0.0f);
```

And in `render()` temporarily force:

```cpp
```

If fog is wired correctly, your workbench + hilt will get a magenta haze with those settings.

> If you STILL see absolutely nothing with magenta fog + MaxDistance=15, then your basic_uniform shaders aren't using the fog uniforms at all (or aren't updated).

C) Confirm you actually updated the *right* shaders

Your compile() loads:

C++

So the fog logic **must** be in **shader/basic_uniform.vert** and **shader/basic_uniform.frag** (not your blade shaders, not older filenames).

The tell-tale sign:

If your `basic_uniform.frag` does **not** contain these uniforms, fog will never work:

glsl

And if your `basic_uniform.vert` does **not** output view-space position to the fragment shader, fog distance can't be computed:

glsl

If those aren't in the correct files, your C++ is setting uniforms that don't exist.

D) One-line debug that instantly proves shader usage

In `basic_uniform.frag`, temporarily do this at the end of `main()`:

glsl

- If your whole scene becomes your fog colour → you're definitely editing/using the correct shader file.
- If nothing changes → you're editing the wrong file OR the program isn't using that shader.

(Then undo it after.)

E) Also: fog won't appear on skybox or blade

- Skybox is drawn with `skyboxShader`, so fog won't touch it.
- Your blade is drawn with `bladeEmissive`, so fog won't touch it either (unless you add fog to that shader too).

So the fog effect should only be visible on:

- the **plane workbench**
- the **hilt mesh**
  (and any other meshes rendered with `prog`)

The most likely culprit in your case

With your current numbers:

```
Fog.MinDistance = 10
```

C++

...and your camera/object distances being roughly around ~10–15 units for what you're looking at, the fog is probably **barely starting** and the colour is **very close to a sky-ish tint** so it looks like "nothing".

That's why the magenta + MaxDistance=15 smoke test is so useful.

If you paste your current **shader/basic_uniform.frag** and **shader/basic_uniform.vert** (the actual files you're compiling), I'll point to the exact line that's preventing fog (or confirm the uniforms are being optimised out).

i still see nothing.

.frag:
#version 460

in vec3 SpotDir;
in vec3 LightDir;
in vec3 ViewDir;
in vec2 TexCoord;
in vec3 ViewPos;

```
layout (location = 0) out vec4 FragColor;
layout (binding = 0) uniform sampler2D DiffuseTex;
layout (binding = 1) uniform sampler2D NormalMapTex;
layout (binding = 2) uniform sampler2D DiffuseTex2;


uniform struct MaterialInfo{
    vec3 Kd;
    vec3 Ka;
    vec3 Ks;
    float Shininess;
}Material;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct FogInfo{
    float MaxDistance;
    float MinDistance;
    vec3 Colour;
}Fog;

//const int levels = 5;
//const float scaleFactor = 1.0/levels;

// Fog controls (simple + robust)
uniform int   FogEnabled;   // 0 = off, 1 = on
uniform float FogScale;     // 1.0 = normal, >1 stronger fog, <1 weaker


vec3 blinnPhong(vec3 n) {
    vec3 diffuse = vec3(0.0f);
    vec3 spec = vec3(0.0f);

    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Light.La * texColour;
    vec3 s = normalize(LightDir);
    //vec3 s = normalize(Light.Position.xyz - position);

    float sDotN = max(dot(s, n), 0.0);

    diffuse = texColour * sDotN;

    if (sDotN > 0.0) {
        vec3 v = normalize(ViewDir);
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }
    return ambient + (diffuse + spec) * Light.L;
}



vec3 blinnPhongSpot(vec3 n)
{
    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Spot.La * texColour;

    vec3 s = normalize(LightDir);  // tangent-space light dir
    vec3 v = normalize(ViewDir);

    float cosAng = dot(normalize(-s), normalize(SpotDir));

    float outer = cos(Spot.Cutoff);                // outer cone edge
    float inner = cos(Spot.Cutoff * 0.85);         // inner cone edge

    float spotEdge = smoothstep(outer, inner, cosAng);
    float spotCore = pow(max(cosAng, 0.0), Spot.Exponent);
    float spotScale = spotEdge * spotCore;

    if (spotScale <= 0.001)
        return ambient;
```

```
    float sDotN = max(dot(s, n), 0.0);
    vec3 diffuse = texColour * sDotN;

    vec3 spec = vec3(0.0);
    if (sDotN > 0.0) {
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }

    return ambient + spotScale * Spot.L * (diffuse + spec);
}

float fogFactorLinear(float dist)
{
    // 1 = fully shaded, 0 = fully fog
    float f = (Fog.MaxDistance - dist) / (Fog.MaxDistance -
Fog.MinDistance);
    return clamp(f, 0.0, 1.0);
}

void main()
{
    vec3 normal = texture(NormalMapTex, TexCoord).xyz;
    normal = normalize(normal * 2.0 - 1.0);

    vec3 litColour = blinnPhongSpot(normal);

    if (FogEnabled != 0)
    {
        // Use view-space distance; scale lets you tune “strength” easily
        float dist = length(ViewPos) * max(FogScale, 0.0001);

        float f = fogFactorLinear(dist);
        vec3 finalColour = mix(Fog.Colour, litColour, f);

        FragColor = vec4(finalColour, 1.0);
    }
    else
    {
        FragColor = vec4(litColour, 1.0);
    }
}

.vert:
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;
layout (location = 2) in vec2 VertexTexCoord;
layout (location = 3) in vec4 VertexTangent;

out vec3 SpotDir;
out vec3 LightDir;
out vec3 ViewDir;
out vec2 TexCoord;
out vec3 ViewPos;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 ProjectionMatrix;
uniform mat4 MVP;

out vec3 LightIntensity, n, pos;

void main()
{
    vec3 N = normalize(NormalMatrix * VertexNormal);
    vec3 T = normalize(NormalMatrix * vec3(VertexTangent.xyz));

    T = normalize(T - N * dot(N, T));

    vec3 B = cross(N, T) * VertexTangent.w;
```

```
    // Columns are T, B, N
    mat3 TBN = mat3(T, B, N);
    mat3 invTBN = transpose(TBN);

    vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    vec3 lightDir;
    if (Light.Position.w == 0.0)
        lightDir = normalize(Light.Position.xyz);       // directional
    else
        lightDir = Light.Position.xyz - Position;       // point light
    LightDir = invTBN * lightDir;

    ViewDir = invTBN * normalize(-Position);
    SpotDir = invTBN * normalize(Spot.Direction);

    TexCoord = VertexTexCoord;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}

.cpp
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    //C1Mesh = ObjMesh::load("media/C1-10P_obj.obj", true);
    //LightsaberMesh = ObjMesh::load("media/Lightsaber_01.obj", true);
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    prog.use();
    glEnable(GL_DEPTH_TEST);

    glEnable(GL_STENCIL_TEST);
    glClearStencil(0);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
```

```
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    //fog
    prog.setUniform("Fog.MinDistance", 0.0f);
    prog.setUniform("Fog.MaxDistance", 15.0f);
    prog.setUniform("Fog.Colour", glm::vec3(1.0f, 0.0f, 1.0f)); // BRIGHT
MAGENTA


    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);


    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");

    //BladeDiffuseTexture =
Texture::loadTexture("media/texture/white.jpg");
    WorkbenchDiffuseMap =
Texture::loadTexture("media/texture/workbench.png");


    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);


}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;   // toggle once
        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
```

```
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();

            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
        //fog stuff
        if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
        {
            fogEnabled = !fogEnabled;   // toggle once
        }
        F_Pressed = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    render();

}

void SceneBasic_Uniform::render()
{

    prog.use();

    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT |
GL_STENCIL_BUFFER_BIT);

    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);


    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    prog.setUniform("FogEnabled", 1);
    prog.setUniform("FogScale", 1.0f);

    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);
    sky.render();




    prog.use();
    //// Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);
```

```
    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
    plane.render();

    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));

    setMatrices();
    LightsaberMesh->render();


    // Blade
    if (bladeOn)
    {
        glm::mat4 bladeBase = mat4(1.0f);
        bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f,
0.0f, 0.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f,
0.0f, 1.0f));

        // -------- Pass 1: White core (writes stencil = 1 where the blade is)
--------
        bladeEmissive.use();

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        // Stencil: always write 1 where fragments pass depth test
        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
        bladeEmissive.setUniform("Intensity", 10.0f);
        bladeEmissive.setUniform("RimPower", 5.0f);
        bladeEmissive.setUniform("RimIntensity", 0.25f);

        bladeEmissive.setUniform("Alpha", 1.0f);
        bladeEmissive.setUniform("CoreAlpha", 1.0f);

        BladeMEsh->render();

        // -------- Pass 2: Coloured glow shell (only OUTSIDE the core
silhouette) --------
        glEnable(GL_BLEND);
        glBlendFunc(GL_ONE, GL_ONE);

        // Don't write depth for glow, but DO depth test so hilt blocks it
        glDepthMask(GL_FALSE);

        // Stencil: only draw where stencil != 1 (i.e., outside the core)
        glStencilMask(0x00);
        glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

        // Optional but usually improves the "wrap" look:
        // render only backfaces of the enlarged shell so it appears as an
outline
        glEnable(GL_CULL_FACE);
        glCullFace(GL_FRONT);

        model = bladeBase;
```

```
        model = glm::scale(model, vec3(0.4f * 1.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        bladeEmissive.setUniform("Intensity", 2.5f);
        bladeEmissive.setUniform("RimPower", 2.0f);
        bladeEmissive.setUniform("RimIntensity", 1.0f);

        bladeEmissive.setUniform("Alpha", 0.55f);
        bladeEmissive.setUniform("CoreAlpha", 0.0f);

        BladeMEsh->render();

        // -------- Restore state --------
        glCullFace(GL_BACK);
        glDisable(GL_CULL_FACE);

        glDepthMask(GL_TRUE);
        glDisable(GL_BLEND);

        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 0, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    }




}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
}

.h
#ifndef SCENEBASIC_UNIFORM_H
#define SCENEBASIC_UNIFORM_H

#include "helper/scene.h"

#include <glm/glm.hpp>
#include <glad/glad.h>
#include "helper/glslprogram.h"
#include "helper/torus.h"
#include "helper/teapot.h"
#include "helper/plane.h"
#include "helper/objmesh.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <array>
#include "helper/cube.h"
#include "helper/skybox.h"


class SceneBasic_Uniform : public Scene
{
private:
    glm::mat4 rotationMatrix;
    SkyBox sky;
    Plane plane;
    Cube cube;
    //std::unique_ptr<ObjMesh> C1Mesh;
    std::unique_ptr<ObjMesh> LightsaberMesh;
    std::unique_ptr<ObjMesh> BladeMEsh;
    glm::mat4 rotateModel;
    //GLuint C1diffuseTexture, C1normalMap;
```

```
    GLuint LSdiffuseTexture, LSnormalMap;
    //GLuint BladeDiffuseTexture;
    GLuint WorkbenchDiffuseMap;
    GLuint cubeTex;


    float tPrev = 0.0f;
    float angle = 0.0f;
    float rotSpeed;
    bool bladeOn = false;
    bool fogEnabled = false;
    float fogScale = 1.0f;
    bool Q_Pressed = false;
    bool E_Pressed = false;
    bool F_Pressed = false;
    glm::vec3 bladeColour;
    std::array<glm::vec3, 6> bladeColours = {
        glm::vec3(0.0f, 0.5f, 1.0f),  // BLUE
        glm::vec3(0.0f, 1.0f, 0.0f),   // GREEN
        glm::vec3(1.0f, 0.35f, 0.0f),   // YELLOW
        glm::vec3(1.0f, 0.0f, 0.0f),   // RED
        glm::vec3(1.0f, 0.0f, 1.0f),   // MAGENTA
        glm::vec3(1.0f, 1.0f, 1.0f),    // WHITE
    };
    int bladeColourIndex = 0;



    GLSLProgram prog, skyboxShader, bladeEmissive;
    void compile();
    void setMatrices();
    void setMatricesEmiss();

    // Camera stuff
    glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    glm::vec3 camFront = glm::normalize(glm::vec3(0.0f, 0.75f, 0.0f) -
camPos);
    glm::vec3 camUp = glm::vec3(0.0f, 1.0f, 0.0f);

    bool firstMouse = true;
    double lastMouseX = 0.0;
    double lastMouseY = 0.0;


    float hiltYawDeg = 0.0f;
    float hiltYawSpeed = 90.0f;
    bool cursorCaptured = true;
    bool escWasDown = false;

public:
    SceneBasic_Uniform();

    void initScene();
    void update( float t );
    void render();
    void resize(int, int);

};

#endif // SCENEBASIC_UNIFORM_H
```

Found it. Your fog maths is fine — **but your `ViewPos` is never being written in the vertex shader**, so in the fragment shader `length(ViewPos)` is effectively garbage/zero and your fog factor ends up ~1 (no fog) all the time.

In your `.vert` you compute:

```glsl
```

...but you **never assign it to `ViewPos`**.

✅ Fix (one line) in `basic_uniform.vert`

Right after that `Position` line, add:

```glsl
```

So that section becomes:

```glsl
```

```
ViewPos = Position;
```

That's the core bug.

Also: your fog starts OFF in C++

In your header you have:

```C++
```

But in `render()` you're currently forcing:

```C++
```

So it *should* still show once `ViewPos` is fixed — but if you go back to using your toggle later, set it to:

```C++
```

(or keep false, but then don't forget to press F).

Quick "make sure it's working" test

After the `ViewPos` fix, keep your current aggressive settings:

- `Fog.MinDistance = 0`
- `Fog.MaxDistance = 15`
- `Fog.Colour = magenta`
- `FogEnabled = 1`

You should *immediately* see a strong magenta haze on the plane and hilt.

One more small gotcha (not required, but good)

Your `bladeColours` array comment has a mistake:

```C++
```

That's actually **orange-ish**, not yellow. Real yellow is `(1,1,0)` — but that's unrelated to fog.

If you make the `ViewPos = Position;` change and still see nothing, the next thing I'd check is whether your `GLSLProgram::setUniform` is failing silently (some wrappers print "uniform not found"). But 99% here: it's the missing assignment.

okay the fog shows up but the toggle does not work whatsoever so the gof is never removed

Cool — that means the shader side is now fine. The toggle not working is 100% because you're **not actually using** `fogEnabled` **when you send the uniform**.

Right now in `render()` you hard-force it every frame:

```C++
```

So even if `fogEnabled` flips in `update()`, you overwrite it back to `1` immediately.

Fix: use the variable (and stop forcing it)

Replace those two lines in `render()` with:

```C++
```

That's the whole fix.

Also: start fog enabled (optional but nicer)

In your `.h` you currently have:

```C++
```

If you want it on by default:

```C++
```

If you keep it `false`, fog will start off and you press **F** to enable.

Minimal corrected sections

`update()` toggle (yours is fine)

```C++
if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
{
    fogEnabled = !fogEnabled;
```

`render()` uniform set (this is what you must change)

```C++
```

If after this it still "won't toggle", the next most common cause is the **F key is being used elsewhere (or not reaching your window)** — but in your code, the forced `FogEnabled = 1` is definitely the reason it never turns off.

okay thank you that works now,
if i want it to show infront of the skybox too how easy would that be?

It's doable, but it's not a "1 line" thing — because your skybox is drawn with a *different shader* and (usually) it's drawn as "infinitely far away", so your current distance-based fog won't naturally affect it unless you add a fog pass to the skybox shader.

You've got two decent options:

Option A (easy + looks right): Fog the skybox in the skybox fragment shader

This is the cleanest for what you asked ("fog in front of the skybox too").

1) Add fog uniforms to `skybox.frag`

```glsl
uniform int FogEnabled;
```

And after you sample the cubemap colour (whatever your variable is), do:

```glsl
vec3 col = skyColour; // from cubemap
if (FogEnabled != 0) {
    col = mix(col, FogColour, clamp(SkyFogAmount, 0.0, 1.0));
```

This applies a constant "haze layer" over the sky, which reads as fog in front of it.

2) Set those uniforms in C++ before drawing the skybox

Right before `sky.render()`:

```C++
skyboxShader.use();
skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
skyboxShader.setUniform("FogColour", glm::vec3(1.0f, 0.0f, 1.0f)); // or Fog Colour if you mirror
```

Pros: simple, stable, visually convincing.
Cons: the skybox fog isn't distance-based (but that's usually fine because the sky is "infinitely far" anyway).

Option B (more "physical", but fiddlier): Depth-based fog as a full-screen post-process

This is the "proper" way if you want fog applied consistently to *everything* including skybox, blade, etc.

You render the scene to an FBO with a depth texture, then do a fullscreen quad and compute fog using the depth buffer.

Pros: affects literally everything consistently.
Cons: more setup (FBO, depth texture, quad shader). More code than you probably want right now.

What I recommend for your project

Go with **Option A**. It's quick and gives the exact illusion you want: fog layer in front of the skybox.

If you paste your current `skybox.frag`, I'll edit it in-place with the exact variable names you're using for the cubemap sample + output.

here is my current skybox.frag:
```
#version 460

layout (binding = 0) uniform samplerCube SkyboxTex;

layout (location = 0) out vec4 FragColour;

in vec3 Vec;

void main() {
   vec3 texColour = texture(SkyboxTex, normalize(Vec)).rgb;
   FragColour = vec4(texColour, 1.0);
}
```
and my current skybox.vert:
```
#version 460

layout (location = 0) in vec3 VertexPosition;

out vec3 Vec;

uniform mat4 MVP;

void main()
{
   gl_Position = MVP * vec4(VertexPosition, 1.0);
   Vec = VertexPosition;
}
```

Yep — this is the "easy" case. Your skybox shader is tiny, so adding a fog layer is straightforward.

1) Replace `skybox.frag` with this (adds fog overlay + toggle)

```glsl
#version 460

layout (binding = 0) uniform samplerCube SkyboxTex;

layout (location = 0) out vec4 FragColour;

in vec3 Vec;

// Fog controls
uniform int  FogEnabled;     // 0/1
uniform vec3 FogColour;      // same as your scene fog colour
uniform float SkyFogAmount;  // 0..1 (how much haze over the skybox)

void main() {
    vec3 texColour = texture(SkyboxTex, normalize(Vec)).rgb;

    if (FogEnabled != 0) {
        texColour = mix(texColour, FogColour, clamp(SkyFogAmount, 0.0, 1.0));
    FragColour = vec4(texColour, 1.0);
}
```

That gives you "fog in front of the skybox" as a controllable haze.

---

2) Set the uniforms in your C++ right before `sky.render()`

In `render()`, you already have:

```
skyboxShader.use();
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
skyboxShader.setUniform("SkyboxTex", 0);
```

C++

ChatGPT 5.2 Thinking

Share

```
skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
```

C++

Better (keep it consistent with your scene fog)

Since you set your scene fog colour in `initScene()`, store it as a member:

In `.h`:

C++

In `initScene()` use it:

C++

And for the skybox:

C++

Notes / gotchas

- This does **not** need changes to `skybox.vert`.
- This fog is **constant haze** over the sky, which is usually what you want (distance-based fog on an "infinite" skybox doesn't really make sense).
- Your blade won't fog unless you also add fog to `blade.frag` or do a post-process approach.

If you want the skybox fog amount to be controlled by the same `FogScale`, you can do:

C++

(or just keep it as a separate tuning knob).

okay so now id like to make it so that the fogcolour is using the same colours as we are using for the blade, here is my blade.frag and blade.very for reference if that helps

```
#version 460

in vec3 vNormalVS;
in vec3 vPosVS;

layout (location = 0) out vec4 FragColor;

// Set from C++
uniform vec3 BladeColour;
uniform float Intensity;
uniform float RimPower;
uniform float RimIntensity;

// New (for glow shell control)
uniform float Alpha;        // overall alpha scale, e.g. 0.15..0.6
uniform float CoreAlpha;    // base alpha even at centre, e.g. 0.0..0.2

void main()
{
    vec3 N = normalize(vNormalVS);
    vec3 V = normalize(-vPosVS); // camera at origin in view-space
```

```
    float ndv = max(dot(N, V), 0.0);
    float rim = pow(1.0 - ndv, RimPower);

    // Emissive colour (core + extra rim pop)
    vec3 col = BladeColour * Intensity;
    col += BladeColour * (rim * RimIntensity * Intensity);

    // Alpha: mostly rim-driven for glow; CoreAlpha lets you keep a faint
fill
    float a = clamp(CoreAlpha + rim * Alpha, 0.0, 1.0);

    FragColor = vec4(col, a);
}

#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;

out vec3 vNormalVS;
out vec3 vPosVS;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 MVP;

void main()
{
    vNormalVS = normalize(NormalMatrix * VertexNormal);
    vPosVS = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Easy — you don't need anything from `blade.vert/frag` for this beyond the fact you already have a **shared colour palette** in C++ (`bladeColours` + `bladeColourIndex`). Just use that palette to drive fog too.

You've got two good behaviours to choose from:

Behaviour 1 (recommended): Fog colour always matches the *current blade colour*

So when you press **E** to cycle blade colour, fog updates too.

1) Add a fog colour member (optional but clean)
In your `.h`, add:

```C++
```

2) Initialise fogColour from the blade palette
In `initScene()` after you set up the blade colour:

```C++
fogColour = bladeColours[bladeColourIndex];
```

3) Update fog colour when you press E (same time you update blade)
In your existing E-press block:

```C++
if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
{
    bladeColourIndex = (bladeColourIndex + 1) % (int)bladeColours.size();

    // Update blade shader
    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour", bladeColours[bladeColourIndex]);

    // Update fog colour to match blade
    fogColour = bladeColours[bladeColourIndex];
    prog.use();
```

```C++
E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
```

4) Make skybox fog use the same fogColour
In `render()` in your skybox section (right before `sky.render()`):

```C++
```

```
skyboxShader.use();
skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
skyboxShader.setUniform("FogColour", fogColour);
skyboxShader.setUniform("SkyFogAmount", skyFogAmount);
```

That's it — fog (scene + skybox) follows the blade colours.

Behaviour 2: Fog colour changes only when fog is toggled on

If you prefer fog colour to "lock in" when you press **F**, just move the `fogColour = bladeColours[...]` line into your F toggle block instead.

Small warning about your palette

Your `bladeColours` array currently has:

```
C++
```

That's not yellow — it's orange-ish. If you want real yellow, use:

```
C++
```

(doesn't affect the wiring, just the look)

If you tell me which behaviour you want (always match vs lock-on-toggle), I'll paste the exact final edited blocks for `initScene()`, `update()`, and `render()` with no extras.

okay thats brill, now if possible, could we also have it so that if the blade is not on, the fog should be a standard greyy colour?

Yep — make fog colour depend on `bladeOn`.

Goal:

- **bladeOn == true** → fogColour = current `bladeColours[bladeColourIndex]`
- **bladeOn == false** → fogColour = grey (and skybox fog matches too)

1) Add these members in your `.h`

```
glm::vec3 fogColour = glm::vec3(0.6f, 0.6f, 0.6f); // default grey
C++
```

2) Add a tiny helper method (recommended)

In your `.h` (private section), declare:

```
C++
```

In your `.cpp`, define it:

```
void SceneBasic_Uniform::updateFogColour()
{
    fogColour = bladeOn ? bladeColours[bladeColourIndex] : fogGrey;

    // Update main scene shader fog colour
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    // Update skybox fog colour too (so it stays consistent)
C++
}
```

3) Call it once during init

At the end of `initScene()` (after shaders are compiled/linked and after your fog uniforms are set):

```
C++
updateFogColour();
```

4) Call it when you toggle blade on/off (Q)

In your Q toggle block:

```cpp
if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS && !Q_Pressed) {
    bladeOn = !bladeOn;
    updateFogColour();
}
Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
```

5) Call it when you change blade colour (E), but only matters if blade is on

In your E block, after updating `bladeColourIndex`:

```cpp
if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
{
    bladeColourIndex = (bladeColourIndex + 1) % (int)bladeColours.size();

    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour", bladeColours[bladeColourIndex]);

    updateFogColour(); // will set blade colour if bladeOn, else stays grey
}
E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
```

6) Keep skybox fog using `fogColour` each frame (safe)

In `render()`, right before `sky.render()`:

```cpp
skyboxShader.use();
skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
skyboxShader.setUniform("FogColour", fogColour);
skyboxShader.setUniform("SkyFogAmount", skyFogAmount);
```

Why this approach is solid

- You only change fog colour when something relevant changes (Q/E), not every frame.
- Blade off instantly snaps fog back to neutral grey.
- Skybox and scene always match.

If you'd rather have the fog *fade* to grey when the blade turns off (instead of snapping), say so and I'll show a simple lerp using `deltaT`.